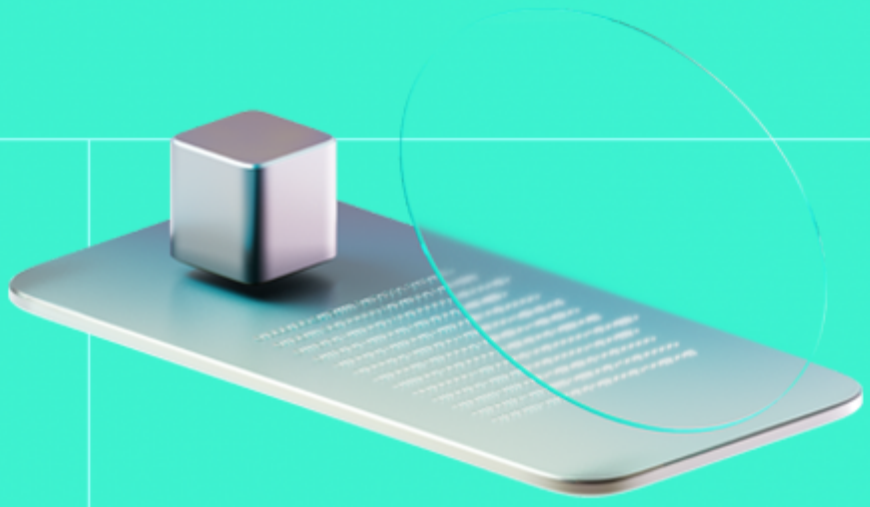
HACKEN

Smart Contract Code Review And Security Analysis Report



Customer: Tokenizer.Estate

Date: 02/12/2025

We express our gratitude to the Tokenizer.Estate team for the collaborative engagement that enabled the execution of this Smart Contract Security Assessment.

Tokenizer aims to be used in different jurisdictions for real estate tokenization.

Document

Name	Smart Contract Code Review and Security Analysis Report for Tokenizer.Estate
Audited By	Turgay Arda Usman, David Camps
Approved By	Ataberk Yavuzer
Website	https://tokenizer.estate/
Changelog	02/12/2025 - Preliminary Report
Platform	Ethereum
Language	Solidity
Tags	ERC20
Methodology	https://docs.hacken.io/methodologies/smart-contracts

Review Scope

Repository	https://github.com/switchcase-devs/tokenizer-contract
Initial Commit	a35daad
Final Commit	8593776

Audit Summary

The system users should acknowledge all the risks summed up in the risks section of the report

8	7	0	1
Total Findings	Resolved	Accepted	Mitigated

Findings by Severity

Severity	Count
Critical	0
High	0
Medium	4
Low	1

Vulnerability	Severity	Status
F-2025-14081 - Minting is Allowed for Frozen and Non-Whitelisted Addresses	Medium	Fixed
F-2025-14099 - Compliance Status is Decoupled from Voting Power	Medium	Fixed
F-2025-14165 - burnFrom Can Brick User Accounts	Medium	Fixed
F-2025-14166 - forceTransfer is Blocked by Pause	Medium	Fixed
F-2025-14167 - Voting Power Desynchronization	Low	Mitigated
F-2025-14078 - Floating Pragma	Info	Fixed
F-2025-14083 - Missing Zero Address Validation	Info	Fixed
F-2025-14084 - Unnecessary Balance Check in _update Consumes Gas	Info	Fixed

Documentation quality

- Functional requirements are partially provided.
  - System requirements is missing.
  - Business logic is missing.
  - USe cases are missing.
- Technical description is provided.

Code quality

- The code mostly follows best practices and style guides:
  - See informational findings for more details.
- The development environment is configured.

Test coverage

Code coverage of the project is **100%** (branch coverage).

- Deployment and basic user interactions are covered with tests.
- Negative cases coverage is provided.
- Interactions by several users are tested thoroughly.

Table of Contents

System Overview

Tokenizer aims to be used in different jurisdictions for real estate tokenization.

RealEstateToken — an upgradeable ERC-20 token with pausing, burning, voting, and permit functionalities.

Privileged roles

- **ROLE_ADMIN**: Enable or disable whitelist mode. Has full authority to grant or revoke any role, including itself, and oversees the hierarchy and administration of all privileged roles within the system.
- **ROLE_WHITELIST**: Add or remove accounts from the whitelist.
- **ROLE_TRANSFER_RESTRICT**: Freeze accounts and lock balances.
- **ROLE_TRANSFER**: Force transfers of tokens from any account.
- **ROLE_MINTER**: Mint new tokens to any account.
- **ROLE_BURNER**: Burn tokens from any account.
- **ROLE_PAUSER**: Pause all state-changing operations (transfers, mints, burns).

Potential Risks

- The project concentrates minting tokens in a single address, raising the risk of fund mismanagement or theft, especially if key storage security is compromised.
- The token contract’s design allows for centralized control over the minting process, posing a risk of unauthorized token issuance, potentially diluting the token value and undermining trust in the project's economic governance.
- The absence of restrictions on state variable modifications by the owner leads to arbitrary changes, affecting contract integrity and user trust, especially during critical operations like minting phases.
- The broad authorization model increases the risk of protocol control loss if any authorized address is compromised, potentially leading to unauthorized actions and significant financial loss.
- The digital contract architecture relies on administrative keys for critical operations. Centralized control over these keys presents a significant security risk, as compromise or misuse can lead to unauthorized actions or loss of funds.
- The token ecosystem grants a single entity the authority to implement upgrades or changes. This centralization of power risks unilateral decisions that may not align with the community or stakeholders' interests, undermining trust and security.
- The project's contracts are upgradable, allowing the administrator to update the contract logic at any time. While this provides flexibility in addressing issues and evolving the project, it also introduces risks if upgrade processes are not properly managed or secured, potentially allowing for unauthorized changes that could compromise the project's integrity and security.
- The `RealEstateToken` is implemented as an ERC20 token with 0 decimals, which is non-standard and can introduce significant precision loss or truncation when performing arithmetic operations involving token amounts, since all values are forced to whole units with no fractional representation.
- The system enforces a strict whitelisting mechanism that blocks token transfers whenever either the sender or the recipient is not whitelisted, meaning that any account holding the `ROLE_WHITELIST` permission can indirectly freeze a user’s funds by removing them from the whitelist via `setWhitelist()`. Because token transfers depend entirely on this whitelist status, users can lose the ability to move or access their tokens at any time, effectively granting the whitelisting authority full control over asset mobility.
- The system includes a transfer restriction mechanism where the account assigned the `ROLE_TRANSFER_RESTRICT` role can call `setFrozen()` to freeze user accounts, preventing them from both sending and receiving tokens. Because frozen status fully disables token mobility, holders of this role effectively gain unilateral control over users’ ability to access or move their assets. This introduces a significant centralization and censorship risk, as accounts can be frozen at will, potentially leading to asset immobility, loss of user autonomy, and reduced trust in the system’s fairness and security guarantees.
- The presence of the `forceTransfer()` function allows the `ROLE_TRANSFER` system role to arbitrarily move tokens from one user to another without requiring prior approval or allowance, effectively bypassing standard ERC20 ownership protections. Since this operation forcibly transfers tokens from users who may not consent or even be aware of the action, the role holder gains sweeping control over all token balances in the system.

This creates a critical centralization and asset-custody risk, as privileged operators can seize, reallocate, or manipulate user funds at will, undermining user trust, violating typical ERC20 security assumptions, and potentially exposing the system to abuse or governance failures.

- The `burnFrom()` function allows an account with the `ROLE_BURNER` to destroy tokens from arbitrary user accounts without requiring allowance or user consent, granting this role unilateral authority to reduce user balances at will. This deviates from standard ERC20 burn mechanics, where `burnFrom()` typically requires prior approval, and introduces a critical centralization and asset-custody risk, as privileged operators can permanently remove user funds without restriction or accountability. Such unrestricted burning capabilities undermine user trust, violate common ERC20 assumptions, and create significant potential for abuse or mismanagement.
- The `lockBalance()` function allows an account with the `ROLE_TRANSFER_RESTRICT` to arbitrarily lock portions of a user's balance, reducing the amount they are able to transfer. Because this mechanism does not require user consent and can be invoked at any time, the role holder can progressively restrict or fully immobilize a user's tokens by locking all unlocked funds. This introduces a significant centralization and censorship risk, as privileged operators can effectively limit or prevent users from accessing or moving their assets, undermining autonomy and potentially enabling abusive or unfair control over token balances.
- The contract includes a pause mechanism that allows the `ROLE_PAUSER` role to halt all state-changing operations, including transfers, mints, and burns. While pausing can be useful for emergency response, it also centralizes significant control in the hands of the designated role holder, who can unilaterally suspend the system's core functionality at any time. This introduces a high operational and availability risk, as an accidental activation, misuse, or compromise of the `ROLE_PAUSER` account would effectively freeze the entire token ecosystem, disrupt user activity, and prevent normal operation until the system is unpaused.
- The system assigns extensive authority to several highly privileged roles—including `ROLE_ADMIN`, `ROLE_TRANSFER`, `ROLE_MINTER`, `ROLE_BURNER`, `ROLE_PAUSER`, `ROLE_WHITELIST`, and `ROLE_TRANSFER_RESTRICT`—each capable of executing critical operations such as forcing transfers, burning tokens, freezing accounts, locking balances, managing whitelists, and pausing the system. As the token implements `ERC20VotesUpgradeable`, these roles also indirectly influence governance voting power, meaning mismanagement can affect not only token balances but also governance outcomes. Because the platform is intended to be operated by external organizations, the security, integrity, and fairness of the system rely on their ability to administer these roles responsibly and safeguard the associated private keys. Any misconfiguration, misuse, or compromise of these privileged roles would grant operators unrestricted control over user assets, system functionality, and governance voting, presenting a severe governance and custodial risk with the potential to cause irreversible asset loss, systemic disruption, or unauthorized interference with user balances or voting outcomes if not managed under strict operational and security best practices.
- The method `setWhitelistMode()` loops through the entire `_holders` array to performs storage updates. As the array grows, the gas required may exceed the block gas limit, causing the function to revert and potentially causing a DoS.

Findings

Vulnerability Details

F-2025-14081 - Minting is Allowed for Frozen and Non-Whitelisted Addresses - Medium

Description:

The `RealEstateToken` contract includes several compliance and control mechanisms, such as freezing accounts (`setFrozen`), locking balances (`lockBalance`), and enforcing a whitelist (`setWhitelistMode`). These are primarily enforced within the overridden `_update` function, which is the central hub for all token balance changes, including transfers, minting, and burning.

The core compliance checks for frozen accounts and whitelisting are incorrectly scoped, causing them to be completely bypassed during minting and burning operations. This allows the `MINTER` role to create new tokens for addresses that are explicitly frozen or not on the whitelist, and allows the `BURNER` role to destroy tokens from a frozen account. This undermines the contract's intended regulatory controls and can lead to tokens becoming permanently stuck in non-compliant wallets.

In `RealEstateToken.sol`, the `_update` function contains the logic to verify if an account is frozen or whitelisted. However, all of these checks are wrapped inside a conditional statement that only executes for regular transfers:

```
function _update(address from, address to, uint256 amount)
internal
override(ERC20Upgradeable, ERC20PausableUpgradeable, ERC20VotesUpgradeable)
{
    if (from != address(0) && to != address(0)) {
        if (!_forceBypass) {
            if (_frozen[from]) revert AccountFrozen(from);
            if (_frozen[to]) revert AccountFrozen(to);

            uint256 fromBal = balanceOf(from);
            if (fromBal < amount) revert ERC20InsufficientBalance(from, fromB
al, amount);
            uint256 unlocked = fromBal - _locked[from];
            if (amount > unlocked) revert LockExceedsUnlocked(from, amount, u
nlocked);
        } else {
```

```
            uint256 fromBal2 = balanceOf(from);
            if (fromBal2 < amount) revert ERC20InsufficientBalance(from, from
Bal2, amount);
        }

        if (whitelistEnabled && !_forceBypass) {
            if (!hasRole(ROLE_TRANSFER, _msgSender())) {
                if (!_whitelisted[from]) revert NotWhitelisted(from);
                if (!_whitelisted[to]) revert NotWhitelisted(to);
            }
        } else if (!whitelistEnabled) {
            if (!hasRole(ROLE_TRANSFER, _msgSender())) {
                revert MissingTransferRole(_msgSender());
            }
        }
    }
super._update(from, to, amount);
```

}

- **Minting:** When `mint` is called, it internally calls `_update(address(0), to, amount)`. Since from is address(0), the condition `from != address(0) && to != address(0)` is `false`, and the entire block containing the freeze and whitelist checks is skipped.
- **Burning:** Similarly, when `burnFrom` is called, it internally calls `_update(account, address(0), amount)`. Since to is address(0), the condition is also false, and the checks are skipped.

This means an address can be frozen via `setFrozen`, but the `MINTER` can still mint new tokens to it. Likewise, if `whitelistEnabled` is true, the `MINTER` can mint to an address that is not on the whitelist. The `BURNER` can also burn tokens from a frozen address.

This can lead to:

- If the whitelist is enabled, the `MINTER` can create tokens for a non-whitelisted address. These tokens are then untransferable, as any attempt by the owner to send them will trigger the whitelist check in `_update`, which will fail.
- The ability to mint tokens to a frozen account (e.g., one identified for sanctions or illicit activity) defeats the purpose of the freezing mechanism. It allows new value to be assigned to a restricted account.

Status: Fixed

Classification

Impact Rate: 3/5

Likelihood Rate: 4/5

Exploitability: Semi-Dependent

Complexity: Simple

Severity: Medium

Recommendations

Remediation: The compliance checks should be restructured within the `_update` function to apply correctly to mint, burn, and transfer operations.

Resolution: Fixed in commit ID `8593776`: the `mint()`, `burn()` and `burnFrom()` methods were updated in order to properly avoid minting and burning tokens from frozen and non-whitelisted addresses.

```
        if (_frozen[to]) revert AccountFrozen(to);
        if (whitelistEnabled && !_whitelisted[to]) revert NotWhitelisted(to);
```

Evidences

PoC

Reproduce:

- test_Mint_To_Frozen_Address
  - Admin first marks Bob as frozen.
  - Calls mint(bob, 1 234) and Bob's balance increases.
- test_Mint_To_Unwhitelisted_Address_While_Whitelist_Enabled
  - Whitelist mode is turned on and only Alice is whitelisted; Bob is excluded.
  - Admin mints 2 345 tokens directly to Bob and balance increases.

PoC Code:

```
function test_Mint_To_Frozen_Address() public {
    // Freeze Bob using ROLE_TRANSFER_RESTRICT (held by admin)
    token.setFrozen(bob, true);

    uint256 pre = token.balanceOf(bob);
    token.mint(bob, 1_234);
    assertEq(token.balanceOf(bob), pre + 1_234);
}
```

```
function test_Mint_To_Unwhitelisted_Address_While_Whitelist_Enabled() public
{
    // Enable whitelist and only whitelist Alice
    token.setWhitelistMode(true);
    token.setWhitelist(alice, true);
    // Bob stays un-whitelisted

    uint256 pre = token.balanceOf(bob);
    token.mint(bob, 2_345);
    assertEq(token.balanceOf(bob), pre + 2_345);
}
```

Results:

```
Ran 1 test for test/RealEstateToken_Locks_And_ForceTransfer.t.sol:RealEstateToken_Locks_ForceTransfer_Test
[PASS] test_Mint_To_Frozen_Address() (gas: 81136)
Suite result: ok. 1 passed; 0 failed; 0 skipped; finished in 5.22ms (408.25µs CPU time)
```

```
Ran 1 test for test/RealEstateToken_Locks_And_ForceTransfer.t.sol:RealEstateToken_Locks_ForceTransfer_Test
[PASS] test_Mint_To_Unwhitelisted_Address_While_Whitelist_Enabled() (gas: 110044)
Suite result: ok. 1 passed; 0 failed; 0 skipped; finished in 1.84ms (125.75µs CPU time)
```

F-2025-14099 - Compliance Status is Decoupled from Voting Power - Medium

Description:

The `RealEstateToken` contract integrates features for on-chain governance using OpenZeppelin's `ERC20VotesUpgradeable`. It also implements several compliance mechanisms, such as freezing accounts (`setFrozen`), locking a portion of a user's balance (`lockBalance`), and enforcing a transfer whitelist which restricts unwhitelisted accounts. These features are intended to give administrators control over token movement to comply with potential real-world regulations.

The contract's compliance mechanisms are completely decoupled from its governance logic. When an administrator freezes an account, locks its balance, or when an account is un-whitelisted while the whitelist is active, these actions only restrict the ability to transfer tokens. The account's voting power, as tracked by the `ERC20VotesUpgradeable` checkpoints, remains unchanged. This can lead to a dangerous governance scenario where accounts that are restricted for compliance reasons can still fully participate in and influence votes.

Voting power in `ERC20VotesUpgradeable` is determined by an account's token balance at a specific block, recorded in a series of checkpoints. These checkpoints are only updated when tokens are moved, specifically through the internal `_transferVotingUnits` function.

The compliance functions operate by modifying separate storage variables and are not linked to the voting module:

- `setFrozen` only updates the `_frozen` mapping.
- `lockBalance` only updates the `_locked` mapping.
- `setWhitelist` only updates the `_whitelisted` mapping.

None of these functions interact with the voting checkpoint system. As a result, an account can be frozen, have its balance locked, or be un-whitelisted, preventing it from transferring tokens, but its voting power remains intact because its balance has not changed. The `_update` function checks these statuses to block transfers, but this gatekeeping happens independently of the governance module.

An account frozen due to a reason, or otherwise restricted could still use its voting power to influence critical governance decisions.

Assets:

- src/RealEstateToken.sol [https://github.com/switchcase-devs/tokenizer-contract]

Status: Fixed

Classification

Impact Rate: 3/5

Likelihood Rate: 3/5

Exploitability: Independent

Complexity: Simple

Severity: Medium

Recommendations

Remediation: The compliance functions must be modified to manually adjust the voting power checkpoints. This can be achieved by calling the internal `_transferVotingUnits` function to effectively move the voting power to or from the zero address when a status change occurs

Resolution: In commit `364e745`, the `_updateVotingPower` function is introduced and used in checkpoints.

F-2025-14165 - burnFrom Can Brick User Accounts - Medium

Description:

The `RealEstateToken` allows the admin (`ROLE_BURNER`) to burn tokens from users. It also implements a locking mechanism (`_locked`) where users cannot transfer more than their balance - locked amount. This invariant is enforced in the `_update` function.

The `burnFrom` function reduces a user's token balance without checking or adjusting their locked balance. This can lead to an arithmetic underflow in subsequent transfers, effectively freezing the user's account and preventing them from moving any remaining unlocked tokens until the deficit is manually corrected.

The `burnFrom` function calls `_burn`, which invokes `_update(account, address(0), amount)`. Inside `_update`, the critical check that ensures `amount <= unlocked` is skipped because the recipient is `address(0)` (burning).

```
function _update(address from, address to, uint256 amount) ... {
   if (from != address(0) && to != address(0) && !_forceBypass) {
       uint256 unlocked = fromBal - _locked[from];
       if (amount > unlocked) revert LockExceedsUnlocked(...);
   }
   // ...
}
```

Consequently, an admin can burn tokens such that `balanceOf(account) < _locked[account]`. When the user later attempts to transfer tokens, `_update` is called with a non-zero to address. The contract attempts to calculate `unlocked = fromBal - _locked[from]`. In Solidity `0.8+`, `fromBal < _locked[from]` causes a panic revert due to underflow.

The same problem also exists in the `burn()` function.

This causes a Denial of Service (DoS) for affected users. Their entire account balance becomes non-transferable ("bricked"). Even if they receive more tokens, they cannot transfer them unless the new balance exceeds the old lock amount. This situation also applies tro the `burn` function.

Assets:

- src/RealEstateToken.sol [https://github.com/switchcase-devs/tokenizer-contract]

Status: Fixed

Classification

Impact Rate: 5/5

Likelihood Rate: 3/5

Exploitability: Semi-Dependent

Complexity: Simple

Severity: Medium

Recommendations

Remediation: Update the `_update` logic or `burnFrom` function to ensure that `_locked[account]` is reduced if it exceeds the new balance after a burn.

Resolution: Fixed in commit ID `8593776`: The code automatically reduces a user's `_locked` amount to match their new balance whenever tokens are burned, preventing the locked amount from ever exceeding the total balance and avoiding arithmetic underflows in future transfers.

```
uint256 bal = balanceOf(account);
uint256 locked = _locked[account];
if (locked > bal) {
    uint256 diff = locked - bal;
    _locked[account] = bal;
    emit BalanceUnlocked(account, diff);
}
```

Evidences

PoC:

Reproduce:

- A user (e.g., User A) has 100 tokens. The Admin then calls lockBalance(User A, 100), meaning the user's entire balance is locked (balance: 100, locked: 100, unlocked: 0). The user correctly cannot transfer any tokens.
- The Admin calls burnFrom(User A, 10). This function reduces the user's token balance to 90 but **fails** to reduce the _locked amount, leaving it at 100 (balance: 90, locked: 100).
- The contract's invariant is now broken because locked > balance. The contract logic for calculating unlocked funds is unlocked = balance - locked.
- User A attempts to transfer just 1 token (or receive tokens and then transfer). The contract attempts to calculate unlocked to

verify the transfer.

- The calculation 90 - 100 causes an **Arithmetic Underflow** (Panic Code 0x11) in Solidity 0.8+. The transaction reverts. The user's account is now permanently "bricked" for all outgoing transfers until an admin intervenes to manually fix the lock amount.

PoC

```
function test_BurnFrom_Breaks_Transfer_Logic() public {
    token.setWhitelistMode(true);
    token.setWhitelist(user1, true);
    token.setWhitelist(user2, true);
    token.transfer(user1, 100);

    // Lock user1 balance
    token.lockBalance(user1, 100);

    vm.startPrank(user1);
    vm.expectRevert(abi.encodeWithSelector(RealEstateToken.LockExceedsUnlocke
d.selector, user1, 1, 0));
    token.transfer(user2, 1);
    vm.stopPrank();

    // Burn 10 tokens from user1
    token.burnFrom(user1, 10);

    assertEq(token.balanceOf(user1), 90);
    assertEq(token.lockedBalanceOf(user1), 100);

    vm.startPrank(user1);
    vm.expectRevert();
    token.transfer(user2, 1);
    vm.stopPrank();

    token.unlockBalance(user1, 10);
    assertEq(token.balanceOf(user1), 90);
    assertEq(token.lockedBalanceOf(user1), 90);

    vm.startPrank(user1);
    vm.expectRevert(abi.encodeWithSelector(RealEstateToken.LockExceedsUnlocke
d.selector, user1, 1, 0));
    token.transfer(user2, 1);
    vm.stopPrank();
}
```

Results:

```
Ran 1 test for test/RealEstateToken_Audit_PoC.t.sol:RealEstateToken_Audit_PoC
[PASS] test_BurnFrom_Breaks_Transfer_Logic() (gas: 369134)
Suite result: ok. 1 passed; 0 failed; 0 skipped; finished in 6.83ms (1.92ms C
PU time)
```

F-2025-14166 - forceTransfer is Blocked by Pause - Medium

Description:

The contract features a forceTransfer function accessible only to the `ROLE_TRANSFER` admin. This function is intended to bypass standard restrictions like locks and whitelists (`_forceBypass = true`) to move funds in exceptional circumstances (e.g., lost keys, legal compliance). The contract also inherits `ERC20PausableUpgradeable`.

The `forceTransfer` function fails if the contract is paused. This defeats the purpose of an emergency administrative override, as the admin cannot rescue funds or rectify state during a security incident (`pause`) without first unpausing the contract, which could expose the contract to further exploitation.

`forceTransfer` calls `_transfer`, which calls `_update`. The `RealEstateToken` overrides `_update` to add custom logic but also calls `super._update` to maintain `ERC20Pausable` functionality.

```
function _update(address from, address to, uint256 amount) internal override(
...) {
    ...
    super._update(from, to, amount);
}
```

`ERC20PausableUpgradeable._update` contains the `whenNotPaused` modifier check. Even though `forceTransfer` sets `_forceBypass = true`, this flag is not recognized by the parent `ERC20PausableUpgradeable` contract. Therefore, if the contract is paused, `super._update` reverts with `EnforcedPause()`.

In an emergency scenario (e.g., a hack or bug discovery), the admin would likely pause the contract. If they then attempt to use `forceTransfer` to rescue stolen funds or move assets to safety, the transaction will revert. The admin is forced to unpause the contract to execute the transfer, potentially allowing an attacker to front-run the rescue operation.

Assets:

- src/RealEstateToken.sol [https://github.com/switchcase-devs/tokenizer-contract]

Status: Fixed

Classification

Impact Rate: 4/5

Likelihood Rate: 4/5

Exploitability: Dependent

Complexity: Simple

Severity: Medium

Recommendations

Remediation: Modify the `_update` override to check for the `_forceBypass` flag before calling `super._update`, or explicitly bypass the pause check.

Resolution: Fixed in commit ID `8593776`: the contract overrides the `paused` function to return `false` specifically when the `forceTransfer` operation is active, which tricks the parent contract's pause check into allowing the transfer even if the contract is actually paused.

```
function paused()
    public
    view
    override(PausableUpgradeable)
    returns (bool)
{
    if (_forceBypass) {
        return false;
    }

    return super.paused();
}
```

Evidences

PoC

Reproduce:

1. The test begins by having the admin call `pause()`. This simulates an emergency situation where the contract is halted to prevent a hack or exploit.
2. The test verifies that a standard user transfer reverts with `EnforcedPause()`, confirming that the contract is effectively frozen for normal operations.
3. The admin attempts to call forceTransfer to move funds from a user to themselves. This represents an emergency recovery action (e.g., seizing stolen funds or recovering lost assets).
4. The forceTransfer call reverts with `EnforcedPause()`.

5. This demonstrates that the administrative override (`forceTransfer`) is incorrectly subject to the same pause restrictions as normal users, rendering it useless for emergency fund recovery while the contract is paused.

PoC:

```
function test_ForceTransfer_Fails_When_Paused() public {
    // Admin pauses the contract
    token.pause();

    // Verify normal transfer fails (as expected)
    vm.startPrank(user1);
    vm.expectRevert(bytes4(keccak256("EnforcedPause()")));
    token.transfer(user2, 10);
    vm.stopPrank();

    bytes memory evidence = bytes("court_order_123");
    vm.expectRevert(bytes4(keccak256("EnforcedPause()")));
    token.forceTransfer(user1, admin, 100, evidence);
}
```

Results:

```
Ran 1 test for test/RealEstateToken_Pause_PoC.t.sol:RealEstateToken_Pause_PoC
[PASS] test_ForceTransfer_Fails_When_Paused() (gas: 84748)
Suite result: ok. 1 passed; 0 failed; 0 skipped; finished in 2.06ms (694.29µs
CPU time)
```

F-2025-14167 - Voting Power Desynchronization - Low

Description:

The contract implements custom voting power logic where `votingPower = balance - locked`. It attempts to manually synchronize this voting power by calling `_updateVotingPower` during transfers, mints, and other operations. `ERC20Votes` logic is also inherited.

The custom voting power synchronization logic is fragile and can become desynchronized from the actual token balance logic, specifically during burnFrom operations or if `_transferVotingUnits` behaves unexpectedly. This can lead to users having incorrect voting power (e.g., 0 votes despite holding unlocked tokens, or retaining votes after tokens are burned).

There are two main desynchronization vectors:

1. **Burning Locked Tokens:** When `burnFrom` is called, it invokes `_burn` (which burns standard ERC20 votes) but does not call `_updateVotingPower`.
    1. If a user has 100 Locked Tokens (0 Votes) and 10 are burned.
    2. ERC20Votes reduces "standard" votes by 10.
    3. The custom logic (balance - locked) is never re-evaluated.
    4. This can lead to an inconsistent internal state or "double counting" of vote reduction depending on the implementation of the underlying Votes library checkpoints.
2. **Redundant & Error-Prone Updates:** The contract manually calls `_updateVotingPower` on every transfer. This function calculates a delta and calls `_transferVotingUnits`. This manual delta management is redundant because `ERC20Votes` already handles vote movement on transfer. Conflicting updates (one automatic from ERC20Votes, one manual from RealEstateToken) waste gas and risk logical errors if the calculations drift apart.

This ca cause users to have more or fewer votes than they are entitled to.

Assets:

- src/RealEstateToken.sol [https://github.com/switchcase-devs/tokenizer-contract]

Status: Mitigated

Classification

Impact Rate: 3/5

Likelihood Rate:	2/5
Exploitability:	Independent
Complexity:	Simple
Severity:	Low

Recommendations

Remediation: Remove the manual `_updateVotingPower` logic entirely. Instead, override the `_getVotingUnits` function to inherently reflect the "unlocked balance" rule. This leverages the robust, standard checkpointing logic of ERC20Votes without manual synchronization.

Resolution: As the code patches the synchronization issues by adding explicit state correction calls (`_updateVotingPower`) after risky operations, instead of refactoring the core voting logic to be inherently consistent. While effective in preventing incorrect vote counts, it relies on manually ensuring these sync calls are never missed in future updates.

F-2025-14078 - Floating Pragma - Info

Description: In Solidity development, the pragma directive specifies the compiler version to be used, ensuring consistent compilation and reducing the risk of issues caused by version changes. However, using a floating pragma (e.g., `^0.8.xx`) introduces uncertainty, as it allows contracts to be compiled with any version within a specified range. This can result in discrepancies between the compiler used in testing and the one used in deployment, increasing the likelihood of vulnerabilities or unexpected behavior due to changes in compiler versions.

The project currently uses floating pragma declarations (`^0.8.20`) in its Solidity contracts. This increases the risk of deploying with a compiler version different from the one tested, potentially reintroducing known bugs from older versions or causing unexpected behavior with newer versions. These inconsistencies could result in security vulnerabilities, system instability, or financial loss. Locking the pragma version to a specific, tested version is essential to prevent these risks and ensure consistent contract behavior.

Status: Fixed

Classification

Impact Rate: 1/5

Likelihood Rate: 5/5

Exploitability: Independent

Complexity: Simple

Severity: Info

Recommendations

Remediation: It is recommended to **lock the pragma version** to the specific version that was used during development and testing. This ensures that the contract will always be compiled with a known, stable compiler version, preventing unexpected changes in behavior due to compiler updates. For example, instead of using `^0.8.xx`, explicitly define the version with `pragma solidity 0.8.20;`.

Before selecting a version, review known bugs and vulnerabilities associated with each Solidity compiler release. This can be done by referencing the official Solidity compiler release notes: Solidity GitHu

b releases or Solidity Bugs by Version. Choose a compiler version with a good track record for stability and security.

Resolution: Fixed in commit ID `364e745`: the pragma version of the `RealEstateToken` contract was locked to `0.8.30`.

F-2025-14083 - Missing Zero Address Validation - Info

Description: In Solidity, the Ethereum address `0x0000000000000000000000000000000000000000` is known as the "zero address". This address has significance because it is the default value for uninitialized address variables and is often used to represent an invalid or non-existent address. The Missing zero address control issue arises when a Solidity smart contract does not properly check or prevent interactions with the zero address, leading to unintended behavior.

Affected functions:

- initializer

Assets:

- src/RealEstateToken.sol [https://github.com/switchcase-devs/tokenizer-contract]

Status: Fixed

Classification

Impact Rate: 1/5

Likelihood Rate: 1/5

Exploitability: Independent

Complexity: Simple

Severity: Info

Recommendations

Remediation: It is strongly recommended to implement checks to prevent the zero address from being set during the initialization of contracts.

Resolution: Fixed in commit ID `364e745`: the following check was introduced into the `initialize()` function to prevent the minting of tokens to the zero address, as well as setting up the system roles to the zero address as well.

```
if (admin_ == address(0)) revert ZeroAddressAdmin();
```

F-2025-14084 - Unnecessary Balance Check in _update Consumes Gas - Info

Description:

The `RealEstateToken` contract overrides the standard `_update` function from OpenZeppelin's `ERC20Upgradeable` to implement a series of custom compliance checks before any token movement. These checks include validating against frozen accounts, locked balances, and an optional whitelist. All standard transfers, mints, and burns eventually call this internal function.

The custom `_update` function includes a manual balance check to ensure the sender has sufficient funds before a transfer. However, this check is redundant because the `super._update` function, which is called at the end of the process, already performs the exact same validation. This duplication of logic serves no purpose and unnecessarily increases the gas cost of every transfer.

Within the `_update` override, the contract explicitly checks the sender's balance before allowing the transfer to proceed. This occurs in two places depending on whether the `_forceBypass` flag is active:

```
function _update(address from, address to, uint256 amount)
internal
override(ERC20Upgradeable, ERC20PausableUpgradeable, ERC20VotesUpgradeable)
{
    if (from != address(0) && to != address(0)) {
        if (!_forceBypass) {
            if (_frozen[from]) revert AccountFrozen(from);
            if (_frozen[to]) revert AccountFrozen(to);

            uint256 fromBal = balanceOf(from);
            if (fromBal < amount) revert ERC20InsufficientBalance(from, fromB
al, amount);
            uint256 unlocked = fromBal - _locked[from];
            if (amount > unlocked) revert LockExceedsUnlocked(from, amount, u
nlocked);
        } else {
            uint256 fromBal2 = balanceOf(from);
            if (fromBal2 < amount) revert ERC20InsufficientBalance(from, from
Bal2, amount);
        }

        if (whitelistEnabled && !_forceBypass) {
            if (!hasRole(ROLE_TRANSFER, _msgSender())) {
                if (!_whitelisted[from]) revert NotWhitelisted(from);
```

```
                if (!_whitelisted[to]) revert NotWhitelisted(to);
            }
        } else if (!whitelistEnabled) {
            if (!hasRole(ROLE_TRANSFER, _msgSender())) {
            revert MissingTransferRole(_msgSender());
        }
    }
    super._update(from, to, amount);
}
```

After these checks, the function calls `super._update(from, to, amount)`. This eventually resolves to the implementation in `ERC20Upgradeable.sol`, which contains its own robust balance check and will revert with `ERC20InsufficientBalance` if the from balance is less than the value. Because the parent contract already provides this safety guarantee, the manual checks within `RealEstateToken` are superfluous.

The impact is purely a matter of efficiency. Every `transfer`, `transferFrom`, `burnFrom`, and `forceTransfer` action will incur a higher gas cost due to the redundant balanceOf call and the subsequent comparison.

Assets:

- src/RealEstateToken.sol [https://github.com/switchcase-devs/tokenizer-contract]

Status: Fixed

Classification

Impact Rate: 1/5

Likelihood Rate: 1/5

Exploitability: Independent

Complexity: Simple

Severity: Info

Recommendations

Remediation: Remove the manual balance checks from the `_update` function in `RealEstateToken.sol`.

Resolution: In commit `364e745`, the the duplicate `balanceOf` check is removed from the `_update` function.

Disclaimers

Hacken Disclaimer

The smart contracts given for audit have been analyzed based on best industry practices at the time of the writing of this report, with cybersecurity vulnerabilities and issues in smart contract source code, the details of which are disclosed in this report (Source Code); the Source Code compilation, deployment, and functionality (performing the intended functions).

The report contains no statements or warranties on the identification of all vulnerabilities and security of the code. The report covers the code submitted and reviewed, so it may not be relevant after any modifications. Do not consider this report as a final and sufficient assessment regarding the utility and safety of the code, bug-free status, or any other contract statements.

While we have done our best in conducting the analysis and producing this report, it is important to note that you should not rely on this report only — we recommend proceeding with several independent audits and a public bug bounty program to ensure the security of smart contracts.

English is the original language of the report. The Consultant is not responsible for the correctness of the translated versions.

As part of Hacken’s ongoing quality assurance process, we may conduct re-audits of select projects. These re-audits are performed independently from the original audit and are intended solely for internal quality control and improvement. Updated reports resulting from such re-audits will be shared privately with the respective clients and may be published on the Hacken website only with their explicit consent.
The sole authoritative source for finalized and most up-to-date versions of all reports remains the Audits section at https://hacken.io/audits/.

Technical Disclaimer

Smart contracts are deployed and executed on a blockchain platform. The platform, its programming language, and other software related to the smart contract can have vulnerabilities that can lead to hacks. Thus, the Consultant cannot guarantee the explicit security of the audited smart contracts.

Appendix 1. Definitions

Severities

When auditing smart contracts, Hacken is using a risk-based approach that considers **Likelihood**, **Impact**, **Exploitability** and **Complexity** metrics to evaluate findings and score severities.

Reference on how risk scoring is done is available through the repository in our Github organization:

hknio/severity-formula

Severity	Description
Critical	Critical vulnerabilities are usually straightforward to exploit and can lead to the loss of user funds or contract state manipulation.
High	High vulnerabilities are usually harder to exploit, requiring specific conditions, or have a more limited scope, but can still lead to the loss of user funds or contract state manipulation.
Medium	Medium vulnerabilities are usually limited to state manipulations and, in most cases, cannot lead to asset loss. Contradictions and requirements violations. Major deviations from best practices are also in this category.
Low	Major deviations from best practices or major Gas inefficiency. These issues will not have a significant impact on code execution.

Potential Risks

The "Potential Risks" section identifies issues that are not direct security vulnerabilities but could still affect the project’s performance, reliability, or user trust. These risks arise from design choices, architectural decisions, or operational practices that, while not immediately exploitable, may lead to problems under certain conditions. Additionally, potential risks can impact the quality of the audit itself, as they may involve external factors or components beyond the scope of the audit, leading to incomplete assessments or oversight of key areas. This section aims to provide a broader perspective on factors that could affect the project's long-term security, functionality, and the comprehensiveness of the audit findings.

Appendix 2. Scope

The scope of the project includes the following smart contracts from the provided repository:

Scope Details	
Repository	https://github.com/switchcase-devs/tokenizer-contract
Initial Commit	a35daa
Final Commit	8593776
Whitepaper	N/A
Requirements	https://github.com/switchcase-devs/tokenizer-contract
Technical Requirements	https://github.com/switchcase-devs/tokenizer-contract

Asset	Type
src/RealEstateToken.sol [https://github.com/switchcase-devs/tokenizer-contract]	Smart Contract

Appendix 3. Additional Valuables

Additional Recommendations

The smart contracts in the scope of this audit could benefit from the introduction of automatic emergency actions for critical activities, such as unauthorized operations like ownership changes or proxy upgrades, as well as unexpected fund manipulations, including large withdrawals or minting events. Adding such mechanisms would enable the protocol to react automatically to unusual activity, ensuring that the contract remains secure and functions as intended.

To improve functionality, these emergency actions could be designed to trigger under specific conditions, such as:

- Detecting changes to ownership or critical permissions.
- Monitoring large or unexpected transactions and minting events.
- Pausing operations when irregularities are identified.

These enhancements would provide an added layer of security, making the contract more robust and better equipped to handle unexpected situations while maintaining smooth operations.

Frameworks and Methodologies

This security assessment was conducted in alignment with recognised penetration testing standards, methodologies and guidelines, including the NIST SP 800-115 – Technical Guide to Information Security Testing and Assessment, and the Penetration Testing Execution Standard (PTES), These assets provide a structured foundation for planning, executing, and documenting technical evaluations such as vulnerability assessments, exploitation activities, and security code reviews. Hacken's internal penetration testing methodology extends these principles to Web2 and Web3 environments to ensure consistency, repeatability, and verifiable outcomes.